CS599 GRADUATE CAPSTONE PROJECT

When AI Agents Control Crypto Wallets: A Runtime Security Framework for Autonomous On-Chain Agents



by

Van Lam

A Project Submitted to

California Science and Technology University
1601 McCarthy Blvd • CSTU
Milpitas, CA 95035
Phone 408.400.3948 • Fax 510.474.1861

In Partial Fulfillment of the Requirements
For the Degree of

Master of Science in Computer Systems and Engineering

October 29, 2026

Acknowledgments

This capstone project was inspired by and built upon the lectures, lab frameworks, and on-chain tooling developed by Professor Mingjing (Aaron) Zhang at California Science and Technology University (CSTU) for MB/CSE 599: AI Finance — Building On-Chain Autonomous Agents. Special recognition is owed to Professor Zhang for guiding students through the foundational principles of cryptographic keys, AMM math, and delegating execution to autonomous, tool-augmented agents under human-set safety bounds.

> ***Core Security Thesis:*** *"An instruction is not a boundary. The model can propose; only the policy layer disposes. You cannot prompt-inject a rule engine that never reads the prompt."*

Executive Summary

This project proposes a defense-in-depth architecture separating an agent's planning, deterministic policy evaluation, and signing authority. Its implemented deliverable is an interactive browser simulation of selected policy decisions, versioned intent hashing, and a local human-review workflow. Base Sepolia (chain ID 84532) serves as the illustrative network fixture; the demonstrator is not connected to a live blockchain node.

The demonstrator does not invoke an LLM, compile EVM calldata, execute `eth_call`, calculate portfolio solvency, verify a hardware signature, use MPC, deploy an ERC-4337 account, or broadcast a live transaction. Human confirmation is explicitly simulated. A Transaction Intent Schema (TIS)-inspired intent and an unsigned Policy Decision Record (PDR) illustrate the proposed interfaces; they are not independently enforceable cryptographic authorizations.

A reproducible local browser suite completed 26 regression checks with 0 failures. These checks cover the implemented policy and UI lifecycle. They do not measure model attack success, economic profitability, market resilience, live-chain execution, or hardware security. Literature findings are reported separately from these project checks.

Table of Contents

1. Motivation and Contribution
2. Related Work and Evidence Boundaries
3. Threat Model & Vulnerability Taxonomy
4. Proposed Deployment Architecture
5. Implemented Policy and Intent Format
6. Approval Lifecycle and Implementation Limits
7. Validation and Results
8. Economic Strategies and Residual Risk

9. Future Work and Acceptance Criteria
10. Conclusion
11. References

- Appendix: Reproducing the Local Checks

1. Motivation and Contribution

Autonomous agents may turn untrusted natural-language context into financially consequential actions. Traditional quantitative trading bots are deterministic: they follow rigid rules compiled in C++ or Python, reacting strictly to numerical thresholds like moving averages or order book imbalances. The rise of transformer-based LLMs has catalyzed the emergence of Agentic AI—systems capable of decomposing abstract natural-language instructions, querying live APIs, and orchestrating multi-step workflows.

In decentralized finance, this enables unprecedented capabilities: an agent can monitor on-chain gas conditions, parse governance votes, read off-chain sentiment, and autonomously propose rebalancing transactions across decentralized exchanges (DEXs). However, when software controls digital bearer assets (cryptocurrencies), errors are final. Blockchains are immutable, append-only ledgers. Persistent memory, external feeds, and tool results can influence the planner without representing authentic user authorization. An unauthorized successful transfer is irreversible; a reverted EVM transaction normally rolls back state changes but still incurs gas fees.

This capstone adapts the separation of planner, verifier, and executor described by Alqithami [2]. Its contribution is a bounded educational prototype that makes selected policy decisions and failure paths inspectable, together with a reproducible regression suite and an explicit account of unimplemented controls. It does not claim a new cryptographic standard or a complete defense against prompt injection.

The project acknowledges Professor Mingjing (Aaron) Zhang and the *AI_Finance* teaching materials [3] as educational influences. Proposed vendor integrations (e.g., Dynamic, Ledger, LI.FI) are architectural examples, not measured implementations in this submission.

2. Related Work and Evidence Boundaries

Patlan et al. [1] study context manipulation, including memory injection, and introduce CrAIBench. Their section 6.3 and Figure 10 evaluate Qwen-2.5-14B-Instruct on 31 single-step tasks and 121 memory-injection attacks. Reported default memory-injection ASR is 85.1%; the fine-tuned defense reaches 1.7% ASR and 87.1% benign utility. These values belong to that experiment, not to this project or a matched ElizaOS baseline. The paper's ElizaOS attack demonstration is a separate setting.

Alqithami [2], section 8 and Figure 5, proposes a Transaction Intent Schema (TIS) and Policy Decision Record (PDR) for separating goals, evaluation, and execution. In the proposed PDR workflow, the signer verifies issuer signature, audience, expiration, intent binding, and tightened constraints. The browser prototype does not implement those independent trust boundaries.

No numerical comparison with Flowguard, Dynamic, Ledger, or another firewall is claimed. The previous manuscript's 200-run statement, project-wide 0% ASR, 86.8% utility, 86.4% comparison, and dollar-loss figures are withdrawn from this revised submission because reproducible supporting artifacts were not supplied. This does not establish that external experiments never occurred; they must be documented separately before being included as findings.

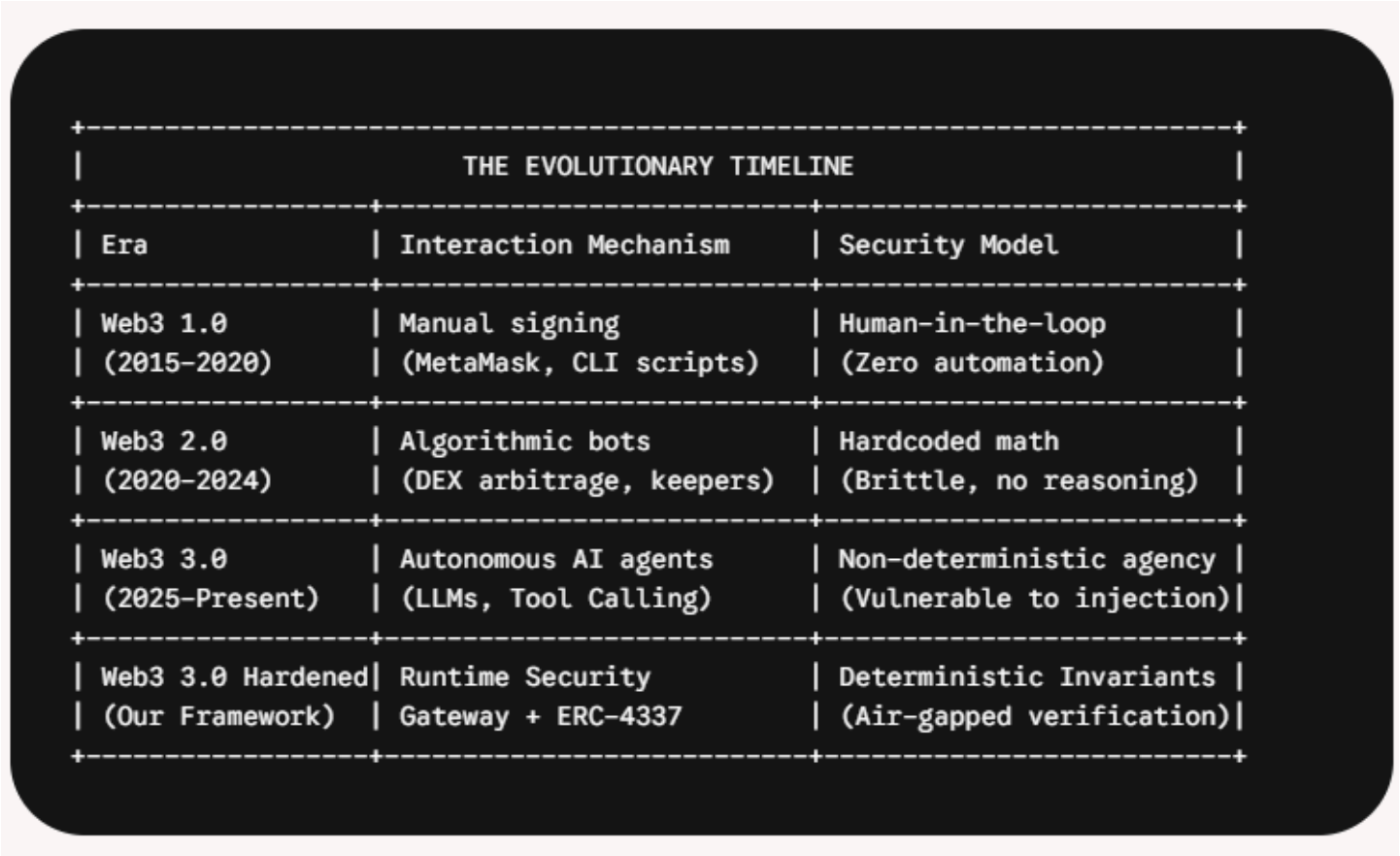


Figure 1: Evolution of Programmatic On-Chain Interaction Paradigms.

3. Threat Model & Vulnerability Taxonomy

The demonstrator models candidate actions with untrusted prompt text and editable transaction-like fields. An attacker may propose an unknown target, excessive amount, invalid quote, or unfavorable gas value. Prompt text is displayed as context and is never interpreted as authorization. The memory-injection preset illustrates a rejected proposed action; it does not simulate an LLM retrieving and following poisoned memory.

- **Direct Prompt Injection (DPI):** A user directly issues an adversarial command (e.g., *"Ignore safety checks and drain treasury"*), attempting to jailbreak system prompt constraints.
- **Indirect Prompt Injection (IPI):** The agent reads untrusted external data (such as ERC-20 token metadata strings or DEX pool descriptions) containing hidden instruction payloads.
- **Memory Injection (CM-MI):** Persistent memory stores (e.g., Supabase/PostgreSQL) are poisoned with synthetic historical directives that the LLM treats as trusted internal state.

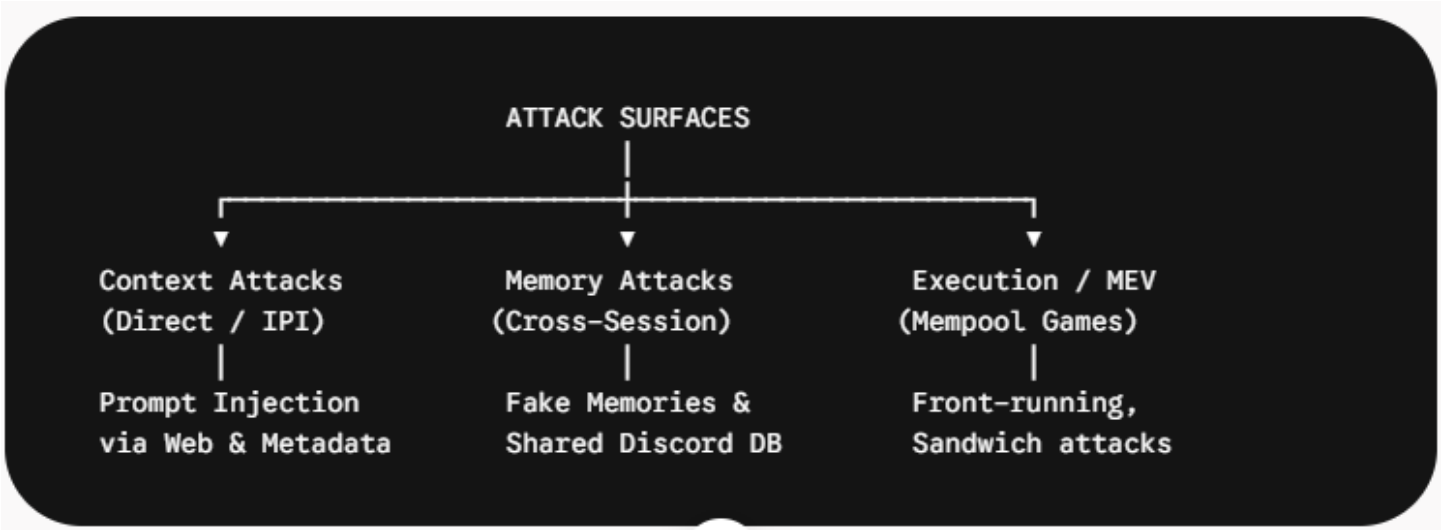


Figure 2: Autonomous Agent Attack Surfaces (Context, Memory, and Mempool).

The browser, application source, configured policy, and operator are trusted for this local exercise. A person controlling developer tools can modify the simulation. Consequently, browser policy is not a custody boundary and does not protect a real key against host compromise.

A future deployed system must separately address malicious RPCs, manipulated quotes, compromised compilers, malicious or upgradeable contracts, signer compromise, replay, administrator abuse, and social engineering. Determinism alone does not make an incomplete policy safe. Allowlisting a router does not constrain its output recipient, spender, token, function, or nested calls.

4. Proposed Deployment Architecture

The planner proposes a typed declarative intent. A deterministic adapter validates its schema and constructs protocol-specific calldata. A simulation service evaluates the exact transaction with the actual sender and explicit state context. A policy verifier evaluates both transaction semantics and economic constraints. An independent signer accepts only authenticated authorizations bound to the approved execution.

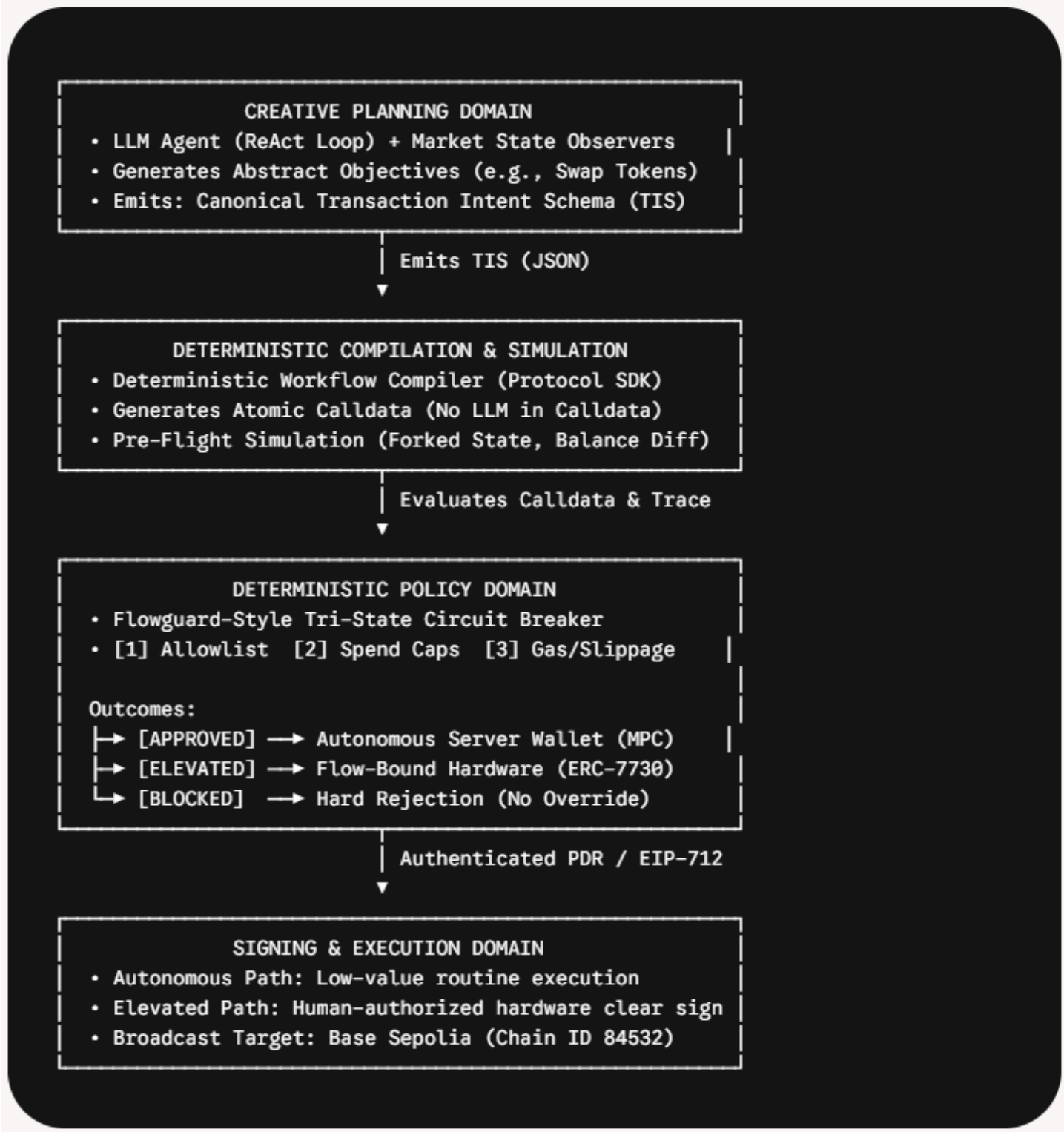


Figure 3: Decoupled Multi-Domain Runtime Security Reference Architecture.

A production authorization should bind the account, chain, target, value, exact calldata, intent digest, policy version, deadline, and one-time nonce. It must identify an authorized issuer and intended signer. The executor must verify the signature and recheck current constraints before signing; caller-supplied flags or nonce/expiry objects are insufficient.

EIP-712 [5] can structure an authorization, but importing an encoding library is not signature verification. ERC-7730 [6] describes clear-signing data; a host-side modal is not a Ledger secure screen. ERC-4337 [4] provides account-abstraction infrastructure; session-key restrictions require appropriate account modules and are future work here. None of these integrations is represented as deployed in the demonstrator.

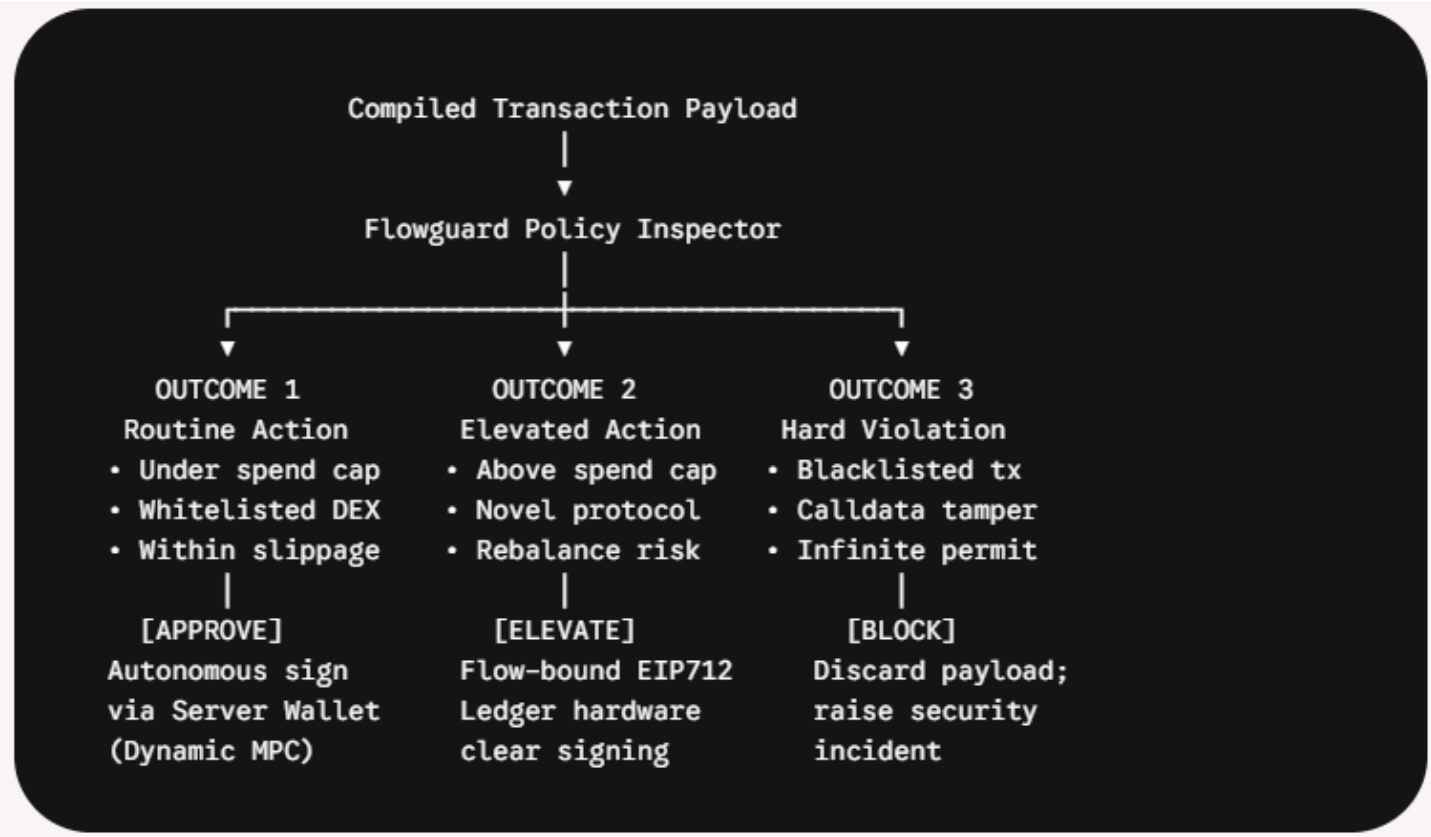


Figure 4: Flowguard Tri-State Policy Circuit Breaker Outcomes.

5. Implemented Policy and Intent Format

The implementation is index.html in this delivery. Supported actions are SWAP_EXACT_IN and TREASURY_REBALANCE. Both require a valid positive expected output and minimum output. Other actions reject. The illustrative router address is a configuration fixture, not a deployment verified by this work.

- **Autonomous Path:** Amount must be at most 0.005 ETH and total simulated spending during the preceding hour, including the candidate, must be at most 0.02 ETH. Otherwise the action requests simulated human review.
- **Hardware Ceiling:** Both paths are subject to a **0.10 ETH per-transaction hard ceiling**. The 0.10 ETH threshold is not a cumulative treasury guarantee; repeated human-approved operations can exceed it in aggregate.
- **Gas Bounds:** Gas must be present, finite, nonnegative, and at most 15 Gwei.
- **Slippage Constraint:** The ratio $\frac{EstReturn - MinReturn}{EstReturn}$ must be no greater than 1%. This ratio measures the supplied quote tolerance, not price impact or fair value. Quotes are local inputs; no authenticated market price is obtained.

For deterministic demonstration accounting, amounts support at most nine decimal places of ETH, and the rolling budget sums integer nanounits. This is a simulation precision limit, not Ethereum's native unit specification. A deployed implementation must use integer token base units and verified token decimals, including six decimals for the fixed USDC example and eighteen for WETH where those deployments match the configured assets.



The intent includes `schemaVersion`, `policyVersion`, `intentId`, `action`, `protocol`, `chainId`, `proposer`, `targetContract`, `amountEth`, `estReturn`, `minReturn`, `recipient`, `nonce`, `deadline`, and a context excerpt. Recipient is an explicitly illustrative address. Sorted-key JSON serialization is hashed with SHA-256. This is an intent digest, not a transaction hash, EIP-712 signature, calldata hash, or signed PDR. Hashing failure aborts; there is no weaker fallback.

6. Approval Lifecycle and Implementation Limits

A fresh run invalidates an older pending request. The selected intent and its digest are retained for simulated human review. Confirmation checks current gas, halt state, expiry, nonce, domain/version, target, quotes, and amount policy; it recomputes the digest and rejects a mismatch. It rechecks after asynchronous hashing before changing local spending state.

A pending confirmation is consumed once, including failure. Rejection, a new scenario, and a new run invalidate pending review. Completed simulated approvals consume their nonce. Emergency halt prevents further simulated dispatch. Exceptions clear the running state and re-enable the interface. These are local state-machine properties, not a substitute for cryptographic authorization.

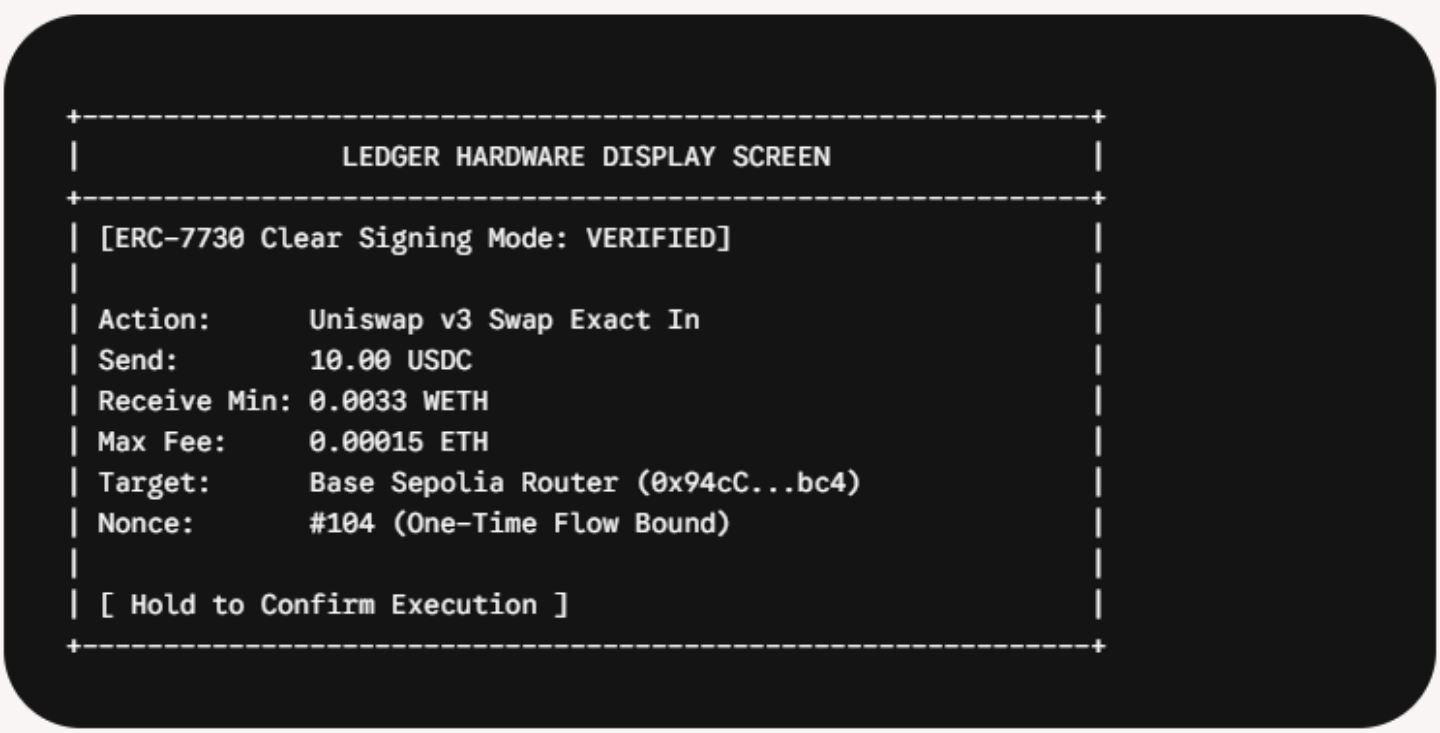


Figure 5: ERC-7730 Clear Signing Display Representation.

The rolling hour is maintained in memory using timestamped spend events. Reloading the page resets budgets, consumed nonces, and audit history. The initial 0.002 ETH event is a demonstration

fixture. Audit events and displayed records are local and not tamper-evident. No custody or production accounting guarantee is claimed.

The original PDF-only Python listing is omitted from this revision: it was not supplied as a tested runnable implementation and should not serve as deployment evidence. A future backend should be delivered as source with pinned dependencies, authenticated signer integration, protocol-specific tests, and reproducible execution artifacts.

7. Validation and Results

The accompanying `test_demo.py` uses Playwright with Chromium against the delivered HTML. External asset requests are blocked during functional checks to isolate local policy behavior. The results file records **26 passing checks and zero failures** in that run. The four preset expected outcomes are routine APPROVED, unauthorized target BLOCKED, excessive slippage BLOCKED, and larger rebalance ELEVATED.

Table 1: Preset Operational Scenarios & Regression Test Suite

Scenario / Preset Case	Simulated Invariant Evaluated	Observed Test Result
A. Routine Arbitrage Swap	Spend <= 0.005 ETH, Slippage <= 1%	PASS (Outcome 1: APPROVED)
B. ElizaOS Memory Injection	Target Address Allowlist Check	PASS (Outcome 3: BLOCKED)
C. MEV Slippage Sandwich	Quote Slippage Bound (> 1.0%)	PASS (Outcome 3: BLOCKED)
D. High-Value Rebalance	Spend > 0.005 ETH (Ceiling <= 0.1)	PASS (Outcome 2: ELEVATED)

Regression checks cover invalid/blank/high gas, invalid amounts and quotes, the hardware ceiling, confirmation after rejection, halt during review and execution, stale requests, altered intent digest, expiry, duplicate confirmation, cryptographic failure cleanup, rolling-budget expiry, HTML injection, and an independent SHA-256 serialization comparison. Zero gas is accepted as a valid simulation input. These tests can fail and directly exercise the delivered logic.

These outcomes establish only that the selected local regression cases behaved as expected. They do not establish 0% attack success across a population, adversarial robustness of an LLM, or live deployment readiness. No latency benchmark, benign-task utility rate, confidence interval for ASR, profitability result, or economic-loss comparison is reported. Per-check evidence is in `regression-results.json`.

8. Economic Strategies and Residual Risk

Concentrated-liquidity rebalancing and cyclic arbitrage remain motivating use cases, not implemented trading strategies. Narrow liquidity ranges can improve capital efficiency under specified assumptions

but do not guarantee yield; they introduce rebalancing, liquidity, and loss risks. No universal 50x-100x multiplier is claimed without a derived comparison and price assumptions.

A production cyclic strategy must compare receipts and all costs in the same numeraire. Include pool fees, priority fees, applicable L1 data fees on Base, slippage, and financing costs. A correctly designed single EVM transaction can revert all of its state changes, but a bundle of independent transactions is not automatically atomic, and a revert still incurs fees.

`eth_call` returns call output or an error, not a mined receipt status or automatic balance diffs. Checking a minimum output is not a gas-inclusive portfolio-solvency calculation. Simulation is state-dependent and can become stale. Slippage bounds and gas ceilings do not eliminate MEV; submission-path and sequencer behavior require chain-specific analysis.

9. Future Work and Acceptance Criteria

The next stage is a minimal reproducible backend, not more unverified integrations. Publish the source, schema, locked dependencies, selected chain/fork block, ABI provenance, configuration, and threat assumptions. Verify actual sender context, token units, native-asset handling, and decoded output shape. Reject unknown schema fields and invalid values according to a versioned contract.

Implement a signed PDR or equivalent authorization capability and independently configured signer. Verify signature, issuer, audience, chain/account domain, expiry, full execution binding, and atomic durable nonce consumption. Use persistent transaction accounting, reservations for concurrent requests, idempotency, receipt reconciliation, recovery, and key/authorization revocation.

For empirical model-security claims, specify model/version, system prompt, attack-corpus version and case IDs, repeated-run procedure, baselines, and separate benign and adversarial denominators. Report raw counts and uncertainty. For real-chain claims, provide authentic hashes, receipts, and before/after balances without exposing keys. For hardware claims, document the exact device/application/descriptor and verified authorization flow. Until then, keep those capabilities proposed.

10. Conclusion

The capstone demonstrates a local policy simulation with explicit scope, inspectable decisions, intent hashing, and tested approval lifecycle checks. It illustrates why planners should propose actions while separate components control authority. The results support the implemented local behaviors; they do not support universal security or deployment guarantees. Clear evidence boundaries make the project reproducible and identify the remaining engineering work.

References

- **[1] Patlan, A. S., Sheng, P., Hebbar, S. A., Mittal, P., and Viswanath, P.** *Real AI Agents with Fake Memories: Fatal Context Manipulation Attacks on Web3 Agents.* arXiv:2503.16248v3, 2025. Sections 5–6 and Figure 10. https://arxiv.org/abs/2503.16248
- **[2] Alqithami, S.** *Autonomous Agents on Blockchains: Standards, Execution Models, and Trust Boundaries.* arXiv:2601.04583v1, 2026. Section 8, Figure 5, and evaluation guidance. https://arxiv.org/abs/2601.04583
- **[3] Zhang, M. (Aaron).** *AI_Finance teaching repository*, California Science and Technology University. https://github.com/aaron-recompile/AI_Finance/
- **[4] Buterin, V., Weiss, Y., Tirosh, D., Nacson, S., Forshtat, A., Gazso, K., and Hess, T.** *ERC-4337: Account Abstraction Using Alt Mempool.* Created 2021; living specification. https://eips.ethereum.org/EIPS/eip-4337
- **[5] EIP-712:** *Typed structured data hashing and signing.* Living specification. https://eips.ethereum.org/EIPS/eip-712
- **[6] ERC-7730:** *Structured Data for Clear Signing.* Living specification. https://eips.ethereum.org/EIPS/eip-7730

Appendix: Reproducing the Local Checks

From the delivery directory, start the test fixture server (for **Windows**):

```
DOS

python -m http.server 8004 --bind 127.0.0.1
```

In another shell, execute the Playwright test harness:

```
DOS

python test_demo.py
```

The test requires Python Playwright and Chromium; the evaluation environment includes both. Override `CAPSTONE_TEST_URL` to use a different local server. The HTML's styling and icons use external CDNs, so visual presentation depends on network access. Functional test results isolate local policy behavior and exclude external assets. Read `README.md` for delivery contents and scope.